

浙江大学



本科实验报告

课程名称:	数据库系统
实验名称:	MiniSql
姓 名:	石耿同
学 号:	3240106167
专 业:	计算机科学与技术
指导教师:	孙建伶
报告日期:	2026/6/25

Table of Contents

一、 工作简述	4
二、 磁盘管理器	4
2.1) 模块功能	4
2.2) 模块设计	5
2.3) 实现细节	6
2.4) 测试与分析	6
三、 缓冲池管理器	7
3.1) 模块功能	7
3.2) 模块设计	8
3.3) 实现细节	10
3.4) 测试与分析	11
四、 记录管理器	12
4.1) 模块功能	12
4.2) 模块设计	12
4.3) 实现细节	13
4.4) 测试与分析	14
五、 目录管理器	14
5.1) 模块功能	14
5.2) 模块设计	15
5.3) 实现细节	15
5.4) 测试与分析	17
六、 索引管理器	17
6.1) 模块功能	17
6.2) 模块设计	18
6.3) 实现细节	19
6.4) 测试与分析	19
七、 执行引擎	20
7.1) 模块功能	20
7.2) 模块设计	21
7.3) 实现细节	22
7.4) 测试与分析	23
八、 恢复管理器	24
8.1) 模块功能	24
8.2) 模块设计	24

8.3) 实现细节	25
8.4) 测试与分析	26

一、工作简述

在本实验中，我负责了 **MiniSQL** 数据库系统中以下七个核心模块的设计与实现：

- **磁盘管理器 (Disk Manager)**: 负责数据库文件的物理存储管理，实现基于 Extent 的页面分配与回收机制，以及逻辑页到物理页的映射。
- **缓冲池管理器 (Buffer Pool Manager)**: 实现内存缓冲池，通过 LRU 替换策略管理页面的缓存、获取、释放与替换，并保证并发安全。
- **记录管理器 (Record Manager)**: 实现基于 Slotted Page 格式的元组存储，支持变长记录的插入、删除、更新，以及表堆 (TableHeap) 的链式页面管理与迭代器遍历。
- **目录管理器 (Catalog Manager)**: 管理数据库的元数据（表和索引的创建、查询、删除），实现 CatalogMeta 的序列化与反序列化，支持数据库重启后的元数据恢复。
- **索引管理器 (Index Manager)**: 基于 B+ 树实现索引的创建、查找、插入、删除和范围扫描，支持节点分裂、合并与重分配，保证树的平衡性。
- **执行引擎 (Execute Engine)**: 实现 SQL 语句的解析分发、执行计划生成与火山模型执行，支持 DDL (CREATE/DROP TABLE/INDEX) 和 DML (SELECT/INSERT/UPDATE/DELETE) 操作。
- **恢复管理器 (Recovery Manager)**: 实现基于 ARIES 算法的日志恢复机制，包含 Redo 阶段和 Undo 阶段，支持事务的原子性和持久性保证。

二、磁盘管理器

2.1) 模块功能

磁盘管理器 (Disk Manager) 负责管理数据库在磁盘上的物理存储。数据库的所有数据存储在单个文件中，采用共享表空间的设计模式。Disk Manager 在整个系统中处于最底层，为上层缓冲池管理器提供页面级的读写和分配接口。

共享表空间

共享表空间的优势在于所有的数据在同一个文件中，方便管理。单个数据库文件主要由记录 (Record)、索引 (Index) 和目录 (Catalog) 组成，在本实验中是一个 .db 文件。

Disk Manager 的核心职责包括：

1. 页面分配与回收：当数据库需要新的存储空间时，在磁盘上分配一个空闲的页面；当页面不再需要时，回收该页面供后续使用。
2. 页面读写：提供从磁盘读取指定页面内容到内存，以及将内存中的页面内容写回磁盘的功能。

3. 逻辑文件层抽象：在物理磁盘文件之上提供基于逻辑页面 ID 的访问接口，上层模块通过逻辑页面 ID 访问数据，无需关心物理存储位置。
4. 元数据持久化：每次页面分配或回收操作后将元数据页 (Meta Page) 立即写回磁盘，保证元数据的一致性。

2.2) 模块设计

2.2.1) 磁盘存储格式

数据库文件采用基于 Extent (区) 的分区组织方式。每个 Extent 由一个位图页和多个数据页组成：

Disk Meta Page	Extent Meta (Bitmap Page)	Extent Pages	Extent Meta (Bitmap Page)	Extent Pages	...
----------------	------------------------------	--------------	------------------------------	--------------	-----

图 1: DiskManager 磁盘存储格式

不同类型的页面

- **Meta Page (元数据页)**: 位于文件物理页 0, 存储整个文件的元数据信息, 包括已分配总页数、Extent 数量、每个 Extent 中已使用页数等, 由 DiskFileMetaPage 类管理。
- **Bitmap Page (位图页)**: 每个 Extent 的第一个页, 用一个比特位标记对应数据页是否空闲 (0 表示空闲, 1 表示已分配), 由 BitmapPage<PAGE_SIZE> 模板类管理。
- **Data Pages (数据页)**: 实际存储数据的页面, 一个 Extent 内数据页的数量由 BITMAP_SIZE 决定。

2.2.2) 核心数据结构

- meta_data_ (char[PAGE_SIZE]): 内存中的元数据页缓存, 存储 DiskFileMetaPage 结构。
- db_io_ (std::fstream): 数据库文件的输入输出流。
- db_io_latch_ (std::recursive_mutex): 文件访问互斥锁, 保证并发安全。

2.2.3) 主要接口

- ReadPage(page_id_t logical_page_id, char *page_data): 根据逻辑页 ID 读取页面内容。
- WritePage(page_id_t logical_page_id, const char *page_data): 将数据写入指定逻辑页。
- AllocatePage(): 分配新页面, 返回逻辑页 ID; 磁盘满时返回 INVALID_PAGE_ID。
- DeAllocatePage(page_id_t logical_page_id): 释放指定逻辑页。
- IsPageFree(page_id_t logical_page_id): 检查逻辑页是否空闲。
- MapPageId(page_id_t logical_page_id): 将逻辑页 ID 映射为物理页 ID。

2.3) 实现细节

2.3.1) 逻辑页到物理页的映射 (MapPageId)

逻辑页 ID 到物理页 ID 的转换是 Disk Manager 的核心算法。每个 Extent 包含 1 个位图页 + BITMAP_SIZE 个数据页：

```
page_id_t DiskManager::MapPageId(page_id_t logical_page_id) {
    uint32_t extent_id    = logical_page_id / BITMAP_SIZE;
    uint32_t page_in_extent = logical_page_id % BITMAP_SIZE;
    // 物理 ID = 元数据页(1) + Extent 大小 * Extent 编号 + 位图页(1) + 区内偏移
    return 1 + extent_id * (BITMAP_SIZE + 1) + 1 + page_in_extent;
}
```

2.3.2) 页面分配 (AllocatePage)

页面分配采用首次适配策略：

1. 检查是否所有 Extent 已满，若满则返回 INVALID_PAGE_ID。
2. 遍历已有 Extent，找到第一个未满足的 Extent（已用页数 < BITMAP_SIZE）。
3. 若所有 Extent 均已满，创建新的 Extent（更新 num_extents_）。
4. 读取对应位图页，调用 BitmapPage::AllocatePage 在位图中分配一个空闲页。
5. 写回位图页，更新 num_allocated_pages_ 和 extent_used_page_[]。
6. 立即将 **Meta Page** 写回磁盘（WritePhysicalPage(META_PAGE_ID, meta_data_)），确保元数据的持久性。

2.3.3) 页面回收 (DeAllocatePage)

1. 计算逻辑页所在的 Extent 和区内偏移。
2. 读取对应位图页，调用 BitmapPage::DeAllocatePage 释放该页。
3. 写回位图页，更新 num_allocated_pages_ 和 extent_used_page_[]。
4. 立即将 **Meta Page** 写回磁盘。

2.3.4) 元数据持久化

一个关键的实现决策是每次页面分配/回收后立即将元数据页写回磁盘。这确保了即使程序异常退出，磁盘上的页面分配状态也是一致的，避免了多个 DiskManager 实例（或重启后）读到过期的元数据。

2.4) 测试与分析

在 test/storage/disk_manager_test.cpp 中有两个主要测试套件：

2.4.1) DiskManagerTest.BitMapPageTest

此测试验证 BitmapPage 的核心功能：初始化、连续分配页面、满容量拒绝分配、单页释放、全部释放后重新分配。

2.4.2) DiskManagerTest.FreePageAllocationTest

此测试直接验证 DiskManager 的完整分配/回收流程：连续分配验证页 ID 连续性，释放特定页面，验证 IsPageFree 的正确性。

2.4.3) 测试结果

```
[-----] 2 tests from DiskManagerTest
[ RUN      ] DiskManagerTest.BitMapPageTest
[          OK ] DiskManagerTest.BitMapPageTest (1 ms)
[ RUN      ] DiskManagerTest.FreePageAllocationTest
[          OK ] DiskManagerTest.FreePageAllocationTest (47055 ms)
[-----] 2 tests from DiskManagerTest (47056 ms total)
```

图 2: DiskManagerTest 测试通过结果

测试名	测试结果
DiskManagerTest.BitMapPageTest	通过✓
DiskManagerTest.FreePageAllocationTest	通过✓

表 1: DiskManager 测试结果

基于测试结果，Disk Manager 的实现正确完成了页面的分配、回收、读写和元数据持久化功能。

三、缓冲池管理器

3.1) 模块功能

缓冲池管理器 (Buffer Pool Manager, BPM) 负责管理内存中的页面缓存区域 (缓冲池)，由一组固定大小的帧 (Frame) 组成。其核心目标是最小化磁盘 I/O，提高数据库性能。主要功能包括：

- 页面缓存：将磁盘页面按需读入内存帧。当上层模块需要访问页面时，BPM 先检查页面是否已在缓冲池中，若在则直接从内存返回。
- 页面获取 (**FetchPage**)：获取指定 `page_id` 的页面。若已在缓冲池中则增加 `pin count` 并返回；若不在则从磁盘读取到可用帧中返回。
- 页面创建 (**NewPage**)：通过 DiskManager 分配新页面，加载到缓冲池可用帧中，返回新页面指针及 `page_id`。
- 页面固定 (**Pin**)：FetchPage 或 NewPage 后页面被固定 (pinned)，`pin count > 0` 时不能被替换。
- 页面释放 (**UnpinPage**)：减少页面的 `pin count`。若页面被修改则设置 `is_dirty = true`。`pin count` 降为 0 后页面成为替换候选。
- 页面替换：缓冲池满时通过 LRUReplacer 选择牺牲帧。被替换的脏页先写回磁盘。

- 页面写回 (**FlushPage**): 将指定页面强制写回磁盘并清除脏标记。
- 页面删除 (**DeletePage**): 从缓冲池移除页面, 通知 `DiskManager` 释放磁盘空间。
- 并发控制: 通过 `recursive_mutex` 保护所有内部数据结构。

3.2) 模块设计

3.2.1) 核心数据结构

- `pages_ (Page[])`: 帧数组, 大小为 `pool_size_`。
- `page_table_ (unordered_map<page_id_t, frame_id_t>)`: 页 ID 到帧 ID 的映射。
- `free_list_ (list<frame_id_t>)`: 空闲帧列表。
- `replacer_ (LRUReplacer*)`: LRU 替换策略。
- `latch_ (recursive_mutex)`: 并发控制锁。

3.2.2) 页面生命周期

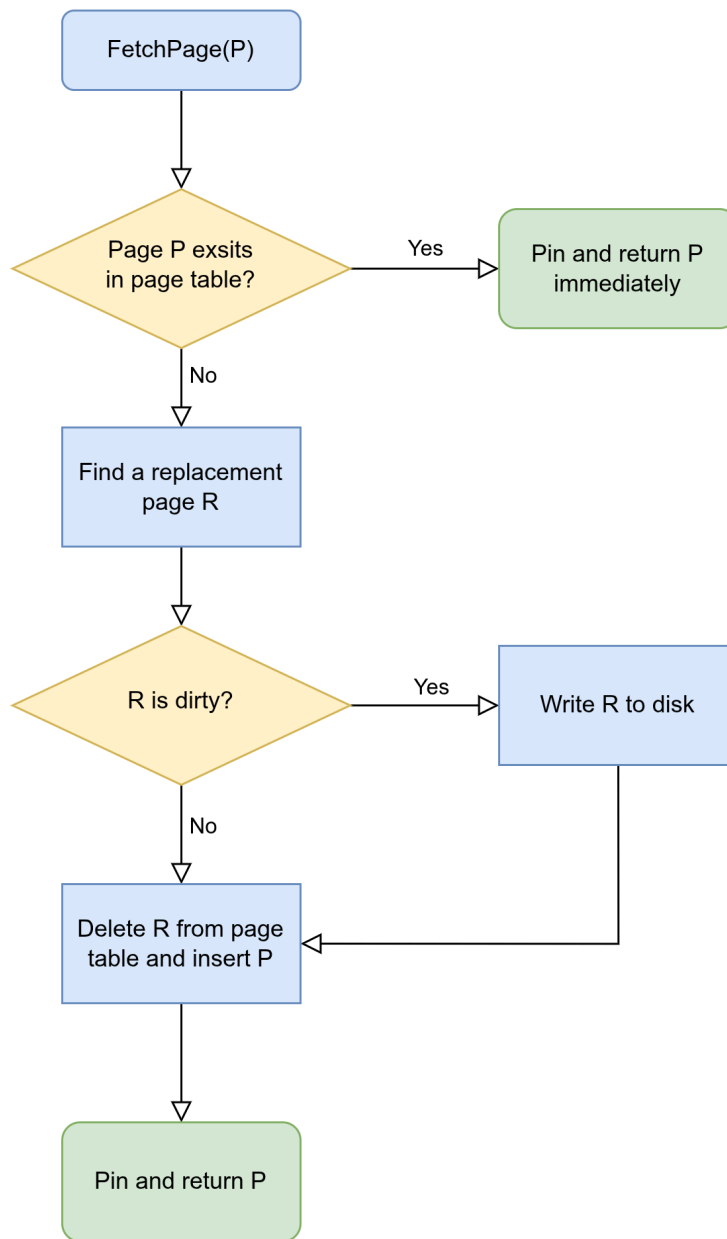


图 3: FetchPage 流程图

3.2.3) 主要接口

- FetchPage(page_id_t): 获取页面，优先从缓冲池命中，否则从磁盘读取。
- NewPage(page_id_t &): 分配新页面并加载到缓冲池。
- UnpinPage(page_id_t, bool is_dirty): 释放页面，减少 pin count。
- FlushPage(page_id_t): 强制写回指定页面。
- DeletePage(page_id_t): 删除页面（释放缓冲池帧和磁盘空间）。
- IsPageFree(page_id_t): 委托给 DiskManager::IsPageFree。

- CheckAllUnpinned(): 调试用，检查所有页面是否已释放。

3.3) 实现细节

3.3.1) FetchPage

```
Page *BufferPoolManager::FetchPage(page_id_t page_id) {
    scoped_lock<recursive_mutex> lock(latch_);
    // 1. 页面已在缓冲池中
    auto it = page_table_.find(page_id);
    if (it != page_table_.end()) {
        frame_id_t fid = it->second;
        replacer_->Pin(fid);    // 从 LRU 移除
        pages_[fid].pin_count++;
        return &pages_[fid];
    }
    // 2. 空闲帧 (先 free_list_ 再 LRU victim)
    frame_id_t fid = TryToFindFreePage();
    if (fid == INVALID_PAGE_ID) return nullptr;
    Page &frame = pages_[fid];
    // 3. 若帧有脏页先写回
    if (frame.is_dirty_) {
        disk_manager_->WritePage(frame.page_id_, frame.data_);
        frame.is_dirty_ = false;
    }
    // 4. 建立新映射，从磁盘读入数据
    if (frame.page_id_ != INVALID_PAGE_ID) page_table_.erase(frame.page_id_);
    page_table_[page_id] = fid;
    frame.page_id_ = page_id;
    frame.pin_count_ = 1;
    frame.is_dirty_ = false;
    disk_manager_->ReadPage(page_id, frame.data_);
    replacer_->Pin(fid);
    return &frame;
}
```

3.3.2) NewPage

与 FetchPage 类似，但调用 DiskManager::AllocatePage() 获取新页号，并使用 memset(frame.data_, 0, PAGE_SIZE) 清零页面内容。

3.3.3) DeletePage

若页面在缓冲池中且 pin count 为 0，从 page_table_ 和 replacer_ 中移除，归还帧到 free_list_；若不在缓冲池中，直接调用 DeallocatePage 释放磁盘空间。

3.3.4) UnpinPage

减少页面的 pin count。若 is_dirty 为 true，置脏标记 (frame.is_dirty_ |= is_dirty)。pin count 归零时通知 replacer_->Unpin(fid) 使该帧成为替换候选。

3.4) 测试与分析

3.4.1) BufferPoolManagerTest.BinaryDataTest

覆盖了 BPM 的核心操作:

- 初始化与新页面创建 (验证 `page_id = 0`)
- 页面读写 (写入随机二进制数据后读出比较)
- 填满缓冲池 (连续 `NewPage` 直到满)
- 缓冲池满时的行为 (全部 `pinned` 时 `NewPage` 应返回 `nullptr`)
- `Unpin`、`Flush` 与替换 (验证 LRU 替换逻辑)
- 获取旧页面 (验证被替换出缓冲池的页面可从磁盘重新读取并保持数据一致)

3.4.2) LRURemplacerTest.SampleTest

验证 LRU 替换策略:

- `Unpin` 操作: 帧加入候选队列
- `Victim` 选择: 选择最久未使用的帧
- `Pin` 操作: 帧从候选队列移除
- 边缘情况: 重复 `Unpin`/`Pin`、空队列 `Victim` 返回 `false`

3.4.3) 测试结果

```
[-----] 1 test from BufferPoolManagerTest
[ RUN      ] BufferPoolManagerTest.BinaryDataTest
[          OK ] BufferPoolManagerTest.BinaryDataTest (16 ms)
[-----] 1 test from BufferPoolManagerTest (16 ms total)
```

图 4: BufferPoolManagerTest.BinaryDataTest 通过结果

```
[-----] 1 test from LRURemplacerTest
[ RUN      ] LRURemplacerTest.SampleTest
[          OK ] LRURemplacerTest.SampleTest (0 ms)
[-----] 1 test from LRURemplacerTest (0 ms total)
```

图 5: LRURemplacerTest.SampleTest 通过结果

测试名	测试结果
BufferPoolManagerTest.BinaryDataTest	通过✓
LRURemplacerTest.SampleTest	通过✓

表 2: BufferPoolManager 及 LRURemplacer 测试结果

基于测试结果, 缓冲池管理器的实现正确完成了页面的获取、创建、替换和释放功能, 验证了 LRU 替换策略的有效性。

四、记录管理器

4.1) 模块功能

记录管理器 (Record Manager) 负责数据库中元组 (记录/行) 的物理存储与访问, 是 SQL 执行层与底层存储之间的关键桥梁。核心功能包括:

- 字段管理 (**Field**): 支持 INT、FLOAT、CHAR 三种数据类型, 处理 NULL 值和变长数据的序列化/反序列化。
- 行管理 (**Row**): 管理一组 Field 的集合, 提供序列化 (写入页面) 和反序列化 (从页面读取) 功能, 支持深拷贝语义。
- 模式管理 (**Schema & Column**): 定义表的列结构, 包含列名、类型、长度、是否可空、是否唯一等元数据。
- **Slotted Page (TablePage)**: 基于 Slotted Page 格式的页面内元组管理, 支持元组的插入、删除、更新和查找。
- 表堆管理 (**TableHeap**): 管理表的页面链表, 支持跨页面的元组操作 (插入、删除、更新、查找)。
- 迭代器 (**TableIterator**): 提供对表堆中元组的顺序遍历能力, 支持 ++ 前置/后置递增操作。

4.2) 模块设计

4.2.1) 行存储格式

Row 序列化格式:

| Field Count (4B) | Null Bitmap (ceil(N/8) B) | Field-1 | ... | Field-N |

Field 序列化格式:

- INT: | 4B value |

- FLOAT: | 4B value |

- CHAR: | 4B length | N bytes data |

4.2.2) Slotted Page 格式

TablePage 采用经典的 Slotted Page 设计:

| HEADER (24B) | ... FREE SPACE ... | ... INSERTED TUPLES ... |

^

free space pointer (向下增长)

Header 包含: PageId、LSN、PrevPageId、NextPageId、FreeSpacePointer、TupleCount。

每个 Tuple Slot 包含: TupleOffset (4B) + TupleSize (4B), 其中 TupleSize 的最高位用作删除标记。

4.2.3) 核心类

- Column: 列定义 (名称、类型、长度、表内索引、可空、唯一)。

- Schema: 表的列集合, 提供列查找和深拷贝/浅拷贝功能。
- Field: 单个字段值, 使用 Union 存储 INT 或 FLOAT 值, CHAR 类型管理独立内存。
- Row: 一行数据, 管理 `vector<Field*>`, 提供序列化/反序列化/深拷贝。
- TablePage: 继承 Page, 实现 Slotted Page 内的元组操作。
- TableHeap: 管理表的页面链表, 提供元组级操作和迭代器。
- TableIterator: 表堆的前向迭代器。

4.3) 实现细节

4.3.1) TablePage::InsertTuple

1. 检查页面剩余空间是否足够 (`GetFreeSpaceRemaining() >= serialized_size + SIZE_TUPLE`)。
2. 先尝试复用已删除的空 slot (`GetTupleSize(i) == 0`), 若没有则使用空闲空间。
3. 更新 FreeSpacePointer, 序列化 Row 到页面数据区。
4. 记录 slot 的 offset 和 size, 设置 Row 的 RowId(page_id, slot_num)。

4.3.2) TablePage::UpdateTuple

1. 检查新行与旧行的序列化大小。若新行过大 (原地更新空间不足), 返回 false 由上层处理 (删除+插入)。
2. 若空间足够, 执行原地更新: memmove 移动数据区域, 更新 FreeSpacePointer 和 slot 元数据。
3. 更新所有受影响的 slot offset (因为 memmove 改变了数据区布局)。

4.3.3) TableHeap::InsertTuple

采用首次适配 (**First Fit**) 策略遍历页面链表:

1. 从 first_page_id_ 开始遍历。
2. 对每个页面 Fetch → WLatch → InsertTuple。若成功则 Unpin(脏) 并返回。
3. 若所有页面都满, 分配新页面 (NewPage), 初始化为链表末尾, 写入新元组。
4. 更新前一个页面的 NextPageId 指针。

4.3.4) TableHeap::UpdateTuple

采用原地更新优先策略:

1. Fetch 目标页面, 先 GetTuple 读取旧行。
2. 尝试 page->UpdateTuple 原地更新。
3. 若原地更新失败 (空间不足), 执行 MarkDelete + ApplyDelete 删旧行, 再 InsertTuple 插入新行。

4.3.5) TableIterator 的拷贝语义

迭代器的拷贝构造函数被正确实现为深拷贝:

```
TableIterator::TableIterator(const TableIterator &other)
: table_heap_(other.table_heap_),
  row_(other.row_ ? new Row(*other.row_) : nullptr),
  txn_(other.txn_) {}
```

这确保了后置递增 (iterator++) 在返回旧迭代器副本时不会产生悬空指针。

4.3.6) TableHeap::FreeTableHeap

遍历页面链表，依次 DeletePage 释放所有数据页，用于 DROP TABLE 时回收磁盘空间。

4.4) 测试与分析

```
[-----] 1 test from TableHeapTest
[ RUN      ] TableHeapTest.TableHeapSampleTest
[         OK ] TableHeapTest.TableHeapSampleTest (1283 ms)
[-----] 1 test from TableHeapTest (1283 ms total)
```

图 6: TableHeap 相关测试通过结果

记录管理器的正确性通过 test/storage/table_heap_test.cpp 中的测试用例验证，覆盖了元组的插入、读取、删除、更新，以及跨页操作和迭代器遍历。

测试名	测试结果
TableHeapTest	通过 

表 3: Record Manager 测试结果

基于测试结果，记录管理器的实现正确完成了元组的物理存储、Slotted Page 管理、跨页操作和迭代器遍历功能。

五、目录管理器

5.1) 模块功能

目录管理器 (Catalog Manager) 是 MiniSQL 的元数据管理中心，负责维护数据库中所有表和索引的目录信息。它提供了创建、查询、删除表和索引的统一接口，并负责将元数据持久化到磁盘，支持数据库重启后的恢复。

核心功能包括：

- 表管理：创建表 (CreateTable)、查询表 (GetTable)、删除表 (DropTable)，维护表名到表 ID 的映射。
- 索引管理：创建索引 (CreateIndex)、查询索引 (GetIndex)、删除索引 (DropIndex)，维护索引名到索引 ID 的映射。
- 元数据持久化：将 CatalogMeta (包含所有表和索引的元数据页映射) 序列化到 Catalog Meta Page (逻辑页 0)，并通过 FlushCatalogMetaPage 立即刷盘。
- 数据库恢复：启动时从磁盘反序列化 CatalogMeta，然后逐一调用 LoadTable 和 LoadIndex 恢复所有表和索引的内存结构。

- 索引回填：创建索引时遍历表中已有数据，将键值对插入新索引。

5.2) 模块设计

5.2.1) 核心数据结构

- `table_names_` (`unordered_map<string, table_id_t>`): 表名 → 表 ID。
- `tables_` (`unordered_map<table_id_t, TableInfo*>`): 表 ID → 表信息。
- `index_names_` (`unordered_map<string, unordered_map<string, index_id_t>>`): 表名 → (索引名 → 索引 ID)。
- `indexes_` (`unordered_map<index_id_t, IndexInfo*>`): 索引 ID → 索引信息。
- `catalog_meta_` (`CatalogMeta*`): 内存中的目录元数据（表/索引与元数据页 ID 的映射）。
- `next_table_id_` / `next_index_id_`: 自增 ID 分配器。

5.2.2) CatalogMeta 序列化格式

| Magic (4B) | TableCount (4B) | IndexCount (4B) | TableEntries (8B * N) | IndexEntries (8B * M) |

每个 TableEntry: (table_id: 4B, page_id: 4B) 每个 IndexEntry: (index_id: 4B, page_id: 4B)

5.2.3) 主要接口

- `CreateTable(table_name, schema, txn, table_info)`: 创建表，分配元数据页，创建 TableHeap，注册到目录。
- `GetTable(table_name, table_info)`: 按表名查找表信息。
- `DropTable(table_name)`: 删除表及其所有索引，释放元数据页和数据页。
- `CreateIndex(table_name, index_name, index_keys, txn, index_info, index_type)`: 创建索引，分配元数据页，回填已有数据，注册到目录。
- `GetIndex(table_name, index_name, index_info)`: 按表名和索引名查找索引信息。
- `DropIndex(table_name, index_name)`: 删除索引，销毁 B+ 树数据，释放元数据页。
- `FlushCatalogMetaPage()`: 将 CatalogMeta 序列化到 Catalog Meta Page 并立即刷盘。
- `LoadTable(table_id, page_id)`: 从元数据页反序列化 TableMetadata，重建 TableInfo。
- `LoadIndex(index_id, page_id)`: 从元数据页反序列化 IndexMetadata，重建 IndexInfo。

5.3) 实现细节

5.3.1) CreateTable

1. 检查表名是否已存在 (`table_names_.find`)。
2. 分配 `table_id = next_table_id++`，通过 `NewPage` 分配元数据页。
3. 创建 TableHeap（分配第一个数据页），创建 TableMetadata 并序列化到元数据页。
4. Unpin 元数据页 (`dirty=true`)，立即 **FlushPage** 刷盘。
5. 创建 TableInfo，更新 `table_names_`、`tables_`、`index_names_`。
6. 更新 CatalogMeta 并 `FlushCatalogMetaPage()`。

5.3.2) CreateIndex

1. 检查表存在、索引名不重复。
2. 将列名数组转换为列下标数组 (key_map)。
3. 分配 index_id = next_index_id++, 通过 NewPage 分配元数据页。
4. 创建 IndexMetadata 并序列化, Unpin + FlushPage。
5. 创建 IndexInfo 并 Init (创建 B+ 树)。
6. 回填已有数据: 遍历 TableHeap 中所有行, 提取索引键, 调用 InsertEntry 插入 B+ 树。
7. 更新 idx_map、indexes_、CatalogMeta, FlushCatalogMetaPage()。

5.3.3) DropTable

1. 先收集并删除表的所有索引 (调用 DropIndex)。
2. 释放表的元数据页 (DeletePage)。
3. 调用 table_info->GetTableHeap()->FreeTableHeap() 释放所有数据页。
4. 删除 TableInfo, 清理 tables_、table_names_。

5.3.4) FlushCatalogMetaPage 的持久化保证

```
dberr_t CatalogManager::FlushCatalogMetaPage() const {  
    auto *meta_page = buffer_pool_manager->FetchPage(CATALOG_META_PAGE_ID);  
    catalog_meta_->SerializeTo(meta_page->GetData());  
    buffer_pool_manager->UnpinPage(CATALOG_META_PAGE_ID, true);  
    buffer_pool_manager->FlushPage(CATALOG_META_PAGE_ID); // 立刷盘  
    return DB_SUCCESS;  
}
```

UnpinPage 后立即调用 FlushPage 确保 Catalog Meta Page 被持久化, 防止多个 DBStorageEngine 实例或重启后读到过期数据。

5.3.5) LoadTable 与 LoadIndex

数据库启动时 (CatalogManager 构造函数的 init=false 分支):

1. 从磁盘读取 Catalog Meta Page, 反序列化 CatalogMeta。
2. 遍历 table_meta_pages_, 调用 LoadTable 反序列化 TableMetadata 并重建 TableHeap 和 TableInfo。
3. 遍历 index_meta_pages_, 调用 LoadIndex 反序列化 IndexMetadata 并重建 IndexInfo 和 B+ 树。
4. 恢复 next_table_id_ 和 next_index_id_。

5.4) 测试与分析

```
[-----] 3 tests from CatalogTest
[ RUN      ] CatalogTest.CatalogMetaTest
[         OK ] CatalogTest.CatalogMetaTest (1 ms)
[ RUN      ] CatalogTest.CatalogTableTest
[         OK ] CatalogTest.CatalogTableTest (163 ms)
[ RUN      ] CatalogTest.CatalogIndexTest
[         OK ] CatalogTest.CatalogIndexTest (139 ms)
[-----] 3 tests from CatalogTest (304 ms total)
```

图 7: CatalogTest 测试通过结果

目录管理器的正确性通过 test/catalog/catalog_test.cpp 中的 CatalogTableTest 和 CatalogIndexTest 验证，覆盖了表的创建/查询/删除、索引的创建/查询/删除，以及重启恢复（通过删除 DBStorageEngine 后重新创建模拟重启）。

测试名	测试结果
CatalogTest.CatalogTableTest	通过✓
CatalogTest.CatalogIndexTest	通过✓

表 4: CatalogManager 测试结果

基于测试结果，目录管理器的实现正确完成了表和索引的元数据管理、持久化和恢复功能。

六、索引管理器

6.1) 模块功能

索引管理器 (Index Manager) 基于 B+ 树实现，为数据库提供高效的索引查找、插入、删除和范围扫描功能。B+ 树是一种平衡的多路搜索树，特别适合磁盘等块存储设备上的高效数据访问。

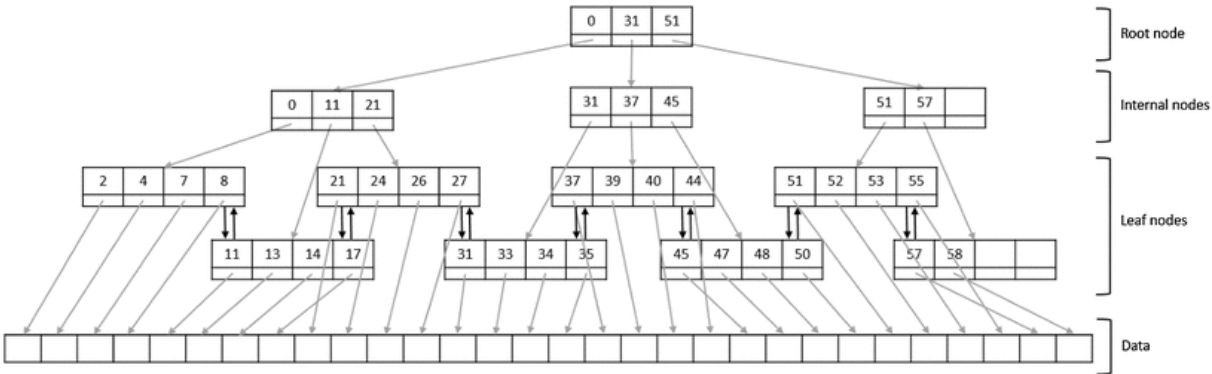


图 8: B+ 树结构示意

核心功能包括:

- 索引创建: 根据 index_id 和键模式构建新的 B+ 树, 在 IndexRootsPage 中注册根页 ID。

- 插入条目 (**InsertEntry**): 将键值对插入 B+ 树, 支持节点分裂并向父节点递归传播。
- 删除条目 (**RemoveEntry**): 从 B+ 树中删除键, 支持节点下溢时的重分配或合并。
- 点查询 (**GetValue**): 精确匹配查找, 返回对应键的所有 RowId。
- 范围扫描 (**ScanKey**): 支持 =、<、<=、>、>=、<> 六种比较操作, 通过迭代器实现高效的范围遍历。
- 迭代器遍历 (**Begin/End**): 提供按键顺序遍历 B+ 树所有条目的迭代器, 支持 ++ 递增。
- 销毁索引 (**Destroy**): 递归释放 B+ 树占用的所有磁盘页面。
- 空树处理: 所有操作在空树 (`root_page_id = INVALID_PAGE_ID`) 时安全返回, 避免空指针解引用。

6.2) 模块设计

索引管理器的核心是 `BPlusTreeIndex` 类 (继承 `Index`), 封装了 `BPlusTree` 的具体实现。`BPlusTree` 类管理树结构, `KeyManager` 处理泛型键的序列化与比较。

6.2.1) 核心类关系

- `Index`: 抽象基类, 定义 `InsertEntry`、`RemoveEntry`、`ScanKey`、`Destroy` 等接口。
- `BPlusTreeIndex`: B+ 树索引实现, 持有 `BPlusTree` 实例和 `KeyManager` (键处理器)。
- `BPlusTree`: B+ 树核心实现, 管理根页、叶子页和内部页的创建、分裂、合并。
- `LeafPage` / `InternalPage`: 叶子节点和内部节点的数据访问接口。
- `IndexIterator`: B+ 树迭代器, 按序遍历所有条目。
- `IndexRootsPage`: 特殊页 (逻辑页 1), 存储所有索引的 (`index_id` → `root_page_id`) 映射。

6.2.2) 关键操作流程

1. 初始化 (**BPlusTree** 构造函数): 从 `IndexRootsPage` 查找当前索引的 `root_page_id`。若为新索引则初始化为 `INVALID_PAGE_ID`, 并在 `IndexRootsPage` 中注册。修改后立即 `FlushPage(INDEX_ROOTS_PAGE_ID)` 持久化。
2. 查找 (**GetValue** → **FindLeafPage**): 从根节点开始, 比较目标键与内部节点中的键, 逐层向下导航直到叶子节点。`GetValue` 先检查 `IsEmpty()` (空树直接返回 `false`), 避免非法页面访问。`FindLeafPage` 支持 `leftMost` 模式 (找最左叶子页)。
3. 插入 (**Insert** → **InsertIntoLeaf** → **InsertIntoParent**): 空树时调用 `StartNewTree` 创建根叶节点。否则找到叶子页插入, 若溢出则分裂节点并将中间键提升到父节点, 递归向上传播 (可能最终分裂根节点增加树高)。
4. 删除 (**Remove** → **CoalesceOrRedistribute**): 空树直接返回。在叶子页中删除键值对, 若下溢 (`size < min_size`) 则尝试从兄弟节点借键 (重分配) 或与兄弟节点合并, 递归向上传播合并/重分配。树变空时 `root_page_id` 重置为 `INVALID_PAGE_ID`, 并通过 `UpdateRootPageId` 更新 `IndexRootsPage`。

5. 范围扫描 (**ScanKey**): 根据不同比较运算符构造对应的迭代器范围, 如 < 操作为 [Begin(), Begin(key)), >= 操作为 [Begin(key), End())。Begin() 和 Begin(key) 均先检查 IsEmpty()。
6. 销毁 (**Destroy**): 先检查 IsEmpty() (空树直接返回), 否则递归遍历树结构, 收集所有页面 ID 后逐一 DeletePage。

6.3) 实现细节

6.3.1) 空树安全处理

所有可能访问根页的方法都增加了 IsEmpty() 检查:

- GetValue: 空树返回 false (等值查询返回空集)。
- Begin() / Begin(key): 空树返回 End() (范围查询返回空集)。
- Destroy: 空树直接返回 (无需释放页面)。
- Remove: 空树记录日志后直接返回。

6.3.2) IndexRootsPage 持久化

BPlusTree 构造函数和 UpdateRootPageId 在修改 IndexRootsPage 后都立即调用 FlushPage(INDEX_ROOTS_PAGE_ID), 确保索引根页映射不会因程序崩溃而丢失。

6.3.3) ScanKey 中 <> 运算符的实现

不等于查询通过“全扫描减去等于”实现: 先用 Begin() 到 End() 收集所有条目, 再用 GetValue 查找等值条目并从结果中移除。

6.4) 测试与分析

B+ 树的正确性通过多个测试文件验证:

6.4.1) BPlusTreeTests.SampleTest

直接针对 BPlusTree 类, 覆盖大量随机数据的插入、查找、删除, 检验节点分裂、合并、重分配等复杂逻辑以及页面管理的正确性。

6.4.2) BPlusTreeTests.BPlusTreeIndexSimpleTest

验证 BPlusTreeIndex 封装类的功能, 包括组合键处理、精确查找和迭代器功能。

6.4.3) BPlusTreeTests.BPlusTreeIndexGenericKeyTest

验证 KeyManager 对多字段组合键 (INT+CHAR 类型) 的序列化和比较正确性。

6.4.4) BPlusTreeTests.IndexIteratorTest

专门验证迭代器在插入和删除操作后的正确性, 确保迭代器正确跳过已删除条目并按键序访问。

6.4.5) 测试结果

```
[-----] 4 tests from BPlusTreeTests
[ RUN      ] BPlusTreeTests.BPlusTreeIndexGenericKeyTest
[          OK ] BPlusTreeTests.BPlusTreeIndexGenericKeyTest (0 ms)
[ RUN      ] BPlusTreeTests.BPlusTreeIndexSimpleTest
I20260625 22:05:47.261740 10590 b_plus_tree_index_test.cpp:58] Insert entry: 0 minisql
I20260625 22:05:47.262801 10590 b_plus_tree_index_test.cpp:58] Insert entry: 1 minisql
I20260625 22:05:47.262878 10590 b_plus_tree_index_test.cpp:58] Insert entry: 2 minisql
I20260625 22:05:47.262914 10590 b_plus_tree_index_test.cpp:58] Insert entry: 3 minisql
I20260625 22:05:47.262956 10590 b_plus_tree_index_test.cpp:58] Insert entry: 4 minisql
I20260625 22:05:47.262975 10590 b_plus_tree_index_test.cpp:58] Insert entry: 5 minisql
I20260625 22:05:47.263007 10590 b_plus_tree_index_test.cpp:58] Insert entry: 6 minisql
I20260625 22:05:47.263046 10590 b_plus_tree_index_test.cpp:58] Insert entry: 7 minisql
I20260625 22:05:47.263073 10590 b_plus_tree_index_test.cpp:58] Insert entry: 8 minisql
I20260625 22:05:47.263108 10590 b_plus_tree_index_test.cpp:58] Insert entry: 9 minisql
[          OK ] BPlusTreeTests.BPlusTreeIndexSimpleTest (68 ms)
[ RUN      ] BPlusTreeTests.SampleTest
[          OK ] BPlusTreeTests.SampleTest (147 ms)
[ RUN      ] BPlusTreeTests.IndexIteratorTest
[          OK ] BPlusTreeTests.IndexIteratorTest (68 ms)
[-----] 4 tests from BPlusTreeTests (283 ms total)
```

图 9: BPlusTree 相关测试通过结果

测试名	测试结果
BPlusTreeTests.SampleTest	通过✓
BPlusTreeTests.BPlusTreeIndexSimpleTest	通过✓
BPlusTreeTests.BPlusTreeIndexGenericKeyTest	通过✓
BPlusTreeTests.IndexIteratorTest	通过✓

表 5: BPlusTree 测试结果

基于测试结果，索引管理器的 B+ 树实现正确处理了插入、删除、查找和范围扫描操作，树结构在各类操作后保持平衡。

七、执行引擎

7.1) 模块功能

执行引擎 (Execute Engine) 是 MiniSQL 的核心调度组件，负责接收解析器生成的抽象语法树 (AST)，生成执行计划并驱动执行器完成用户请求。它与解析器 (Parser)、计划器 (Planner) 以及底层存储引擎紧密协作。

执行引擎概述

执行引擎在系统中扮演“总指挥”的角色：对于 DDL 语句（CREATE/DROP TABLE/INDEX 等）直接处理；对于 DML 语句（SELECT/INSERT/UPDATE/DELETE）则通过计划器生成执行计划，再用火山模型驱动执行器完成操作。

核心功能包括：

- **SQL 命令分发**：根据 AST 节点类型，将 DDL 和 DML 命令分发给对应处理函数。
- **DDL 操作执行**：处理 CREATE/DROP DATABASE、CREATE/DROP TABLE、CREATE/DROP INDEX、SHOW DATABASES/TABLES/INDEXES、USE DATABASE 等命令。
- **DML 操作执行**：通过 Planner 生成执行计划，创建执行器并通过火山模型（Iterator Model）执行。
- **执行上下文管理**：维护 ExecuteContext，包含事务、目录管理器和缓冲池管理器的引用。
- **结果输出**：将 SELECT 查询结果格式化为表格，记录并显示执行耗时。
- **EXECFILE 支持**：从外部文件读取并批量执行 SQL 命令。
- **数据库持久化加载**：启动时扫描 ./databases/ 目录，加载已有数据库。

7.2) 模块设计

7.2.1) 核心组件

- **ExecuteEngine**：主控类，持有 dbs_（数据库名 → DBStorageEngine* 映射）和 current_db_（当前数据库名）。
- **ExecuteContext**：执行上下文，封装 Txn*、CatalogManager*、BufferPoolManager*。
- **Planner**：计划器，将 AST 转换为执行计划（SeqScanPlanNode 或 IndexScanPlanNode）。
- **AbstractExecutor**：执行器基类，派生类包括 SeqScanExecutor、IndexScanExecutor、InsertExecutor、DeleteExecutor、UpdateExecutor、ValuesExecutor。

7.2.2) 主要接口与执行流程

1. 构造函数 **ExecuteEngine(bool init)**：创建 ./databases/ 目录（若不存在）；若 init=false，扫描目录下所有数据库文件并用 init=false 打开，加载到 dbs_ 中。
2. **Execute(pSyntaxNode ast)**：主入口。检查当前数据库是否已选择，创建 ExecuteContext；对 DDL 命令直接分发到对应处理函数；对 DML 命令通过 Planner 生成计划后调用 ExecutePlan 执行。
3. **ExecutePlan(plan, result_set, txn, exec_ctx)**：工厂方法创建执行器 → Init() → 循环 Next() → 收集结果。使用 try-catch 捕获执行异常。
4. **CreateExecutor(exec_ctx, plan)**：根据计划类型创建对应执行器。对于 Update、Delete、Insert 计划，递归创建子执行器。

7.2.3) 执行器设计

- **SeqScanExecutor**：全表扫描，通过 TableIterator 遍历 TableHeap 所有行，对每行应用谓词过滤。支持 Schema 不匹配时的 TupleTransfer（投影）。

- **IndexScanExecutor**: 索引扫描, 遍历可用索引, 调用 `Index->ScanKey()` 收集 RowId 集合 (对 AND 谓词取交集), 再通过 `TableHeap->GetTuple()` 获取行数据。
- **InsertExecutor**: 插入行到 TableHeap, 同时更新所有相关索引 (通过 `InsertEntry`)。
- **DeleteExecutor**: 从 TableHeap 删除行, 同时从所有索引中移除 (通过 `RemoveEntry`)。
- **UpdateExecutor**: 获取旧行, 生成新行 (`GenerateUpdatedTuple`), 调用 `UpdateTuple`, 并更新索引 (先 `RemoveEntry` 旧键, 再 `InsertEntry` 新键)。
- **ValuesExecutor**: 提供原始值行 (用于 INSERT 的来源数据)。

7.3) 实现细节

7.3.1) 计划器 (Planner::PlanSelect)

```
AbstractPlanNodeRef Planner::PlanSelect(shared_ptr<SelectStatement> statement) {
    auto out_schema = MakeOutputSchema(statement->column_list_);
    vector<IndexInfo *> indexes;
    context_>GetCatalog()->GetTableIndexes(statement->table_name_, indexes);
    // 筛选可用索引: 单列索引, 且索引列在 WHERE 条件中
    for (auto index : indexes) {
        if (index->GetIndexKeySchema()->GetColumns().size() == 1) {
            auto col_id = index->GetIndexKeySchema()->GetColumn(0)->GetTableInd();
            if (col_id in statement->column_in_condition_) {
                available_index.push_back(index);
            }
        }
    }
    // 无可用索引或含 OR → SeqScan ; 否则 → IndexScan
    if (available_index.empty() || statement->has_or) {
        return make_shared<SeqScanPlanNode>(out_schema, table_name, where);
    }
    return make_shared<IndexScanPlanNode>(out_schema, table_name, available_index,
        available_index.size() != column_in_condition_.size(),
        where);
}
```

计划器仅在存在单列索引且 WHERE 条件包含该列时才选择索引扫描 (含 `need_filter_` 标记是否需要额外过滤)。若含 OR 则退回全表扫描。

7.3.2) ExecuteCreateTable 中的约束处理

创建表时自动为 PRIMARY KEY 列创建联合索引 (`pk_<表名>`), 为每个标记为 UNIQUE 的列 (且非 PK 列) 创建唯一索引 (`uk_<表名>_<列名>`)。

7.3.3) ExecuteCreateIndex 中的索引回填

创建索引后, 遍历 TableHeap 中所有已有行, 提取索引键并插入 B+ 树, 确保新索引覆盖历史数据。

7.3.4) ExecuteDropTable 的级联删除

删除表时先收集并删除该表的所有索引，再释放表的数据页和元数据页。

7.3.5) 空数据库保护

当用户未选择数据库（current_db_ 为空）时执行需要数据库的查询，会输出 "No database selected." 而非段错误崩溃。

7.3.6) 错误处理

所有 DDL 函数通过 dberr_t 返回状态码，通过 ExecuteInformation() 转换为用户可读消息。DML 执行过程中通过 try-catch 捕获异常。

7.4) 测试与分析

执行引擎的正确性通过 test/execution/executor_test.cpp 中的四个测试用例验证：

- SimpleSeqScanTest: 验证全表扫描和谓词过滤（WHERE id < 500）。
- SimpleRawInsertTest: 验证单行插入及后续扫描确认。
- SimpleUpdateTest: 验证单行更新（SET name = "minisql" WHERE id = 500）。
- SimpleDeleteTest: 验证单行删除（DELETE WHERE id = 50）及索引同步删除。

7.4.1) 测试结果

```
[-----] 4 tests from ExecutorTest
[ RUN      ] ExecutorTest.SimpleSeqScanTest
[      OK  ] ExecutorTest.SimpleSeqScanTest (476 ms)
[ RUN      ] ExecutorTest.SimpleDeleteTest
[      OK  ] ExecutorTest.SimpleDeleteTest (455 ms)
[ RUN      ] ExecutorTest.SimpleRawInsertTest
[      OK  ] ExecutorTest.SimpleRawInsertTest (359 ms)
[ RUN      ] ExecutorTest.SimpleUpdateTest
[      OK  ] ExecutorTest.SimpleUpdateTest (362 ms)
[-----] 4 tests from ExecutorTest (1654 ms total)
```

图 10: ExecutorTest 四个测试全部通过

测试名	测试结果
ExecutorTest.SimpleSeqScanTest	通过✓
ExecutorTest.SimpleRawInsertTest	通过✓
ExecutorTest.SimpleUpdateTest	通过✓
ExecutorTest.SimpleDeleteTest	通过✓

表 6: ExecuteEngine 测试结果

基于测试结果，执行引擎正确实现了 SQL 语句的分发、计划生成和火山模型执行，以及 DDL 操作的表/索引管理。

八、恢复管理器

8.1) 模块功能

恢复管理器 (Recovery Manager) 实现了基于 ARIES (Algorithm for Recovery and Isolation Exploiting Semantics) 算法的日志恢复机制。它通过 Write-Ahead Logging (WAL) 保证事务的原子性和持久性, 能够在系统崩溃后将数据库恢复到一致性状态。

核心功能包括:

- 日志记录管理: 定义了六种日志类型 (Insert、Delete、Update、Begin、Commit、Abort), 每条日志包含 LSN (日志序列号)、prev_lsn (同一事务的前一条日志 LSN)、事务 ID 及操作相关数据。
- **Checkpoint** 恢复: 从 Checkpoint 恢复数据库的持久化快照和活跃事务表。
- **Redo** 阶段: 从 Checkpoint 之后按 LSN 顺序重放所有日志, 重现历史操作。
- **Undo** 阶段: 对未提交的活跃事务按 LSN 逆序回滚, 撤销未完成的操作。
- **UndoTxn**: 沿 prev_lsn 链逆向回滚单个事务的所有日志 (Insert → 删除数据, Delete → 恢复旧值, Update → 恢复旧值)。

8.2) 模块设计

8.2.1) 核心数据结构

- LogRec: 日志记录, 包含类型、LSN、prev_lsn、txn_id 和操作相关数据 (insert key/value、delete key/value、old/new key/value)。
- CheckPoint: 检查点, 包含 checkpoint_lsn_ (持久化的最后一个 LSN)、active_txns_ (活跃事务表)、persist_data_ (持久化数据快照)。
- RecoveryManager: 恢复管理器, 持有 log_recs_ (有序日志映射)、persist_lsn_、active_txns_、data_ (所有数据)。
- LogRecPtr = shared_ptr<LogRec>。

8.2.2) 日志记录类型

类型	含义	关键字段
kBegin	事务开始	txn_id, lsn
kInsert	插入操作	ins_key, ins_val
kDelete	删除操作	del_key, del_val (保存旧值用于 Undo)
kUpdate	更新操作	old_key, old_val, new_key, new_val
kCommit	事务提交	txn_id, lsn
kAbort	事务回滚	txn_id, prev_lsn (用于 Undo)

8.2.3) 恢复流程

Init(CheckPoint)

|

▼

RedoPhase() ← 从 persist_lsn_ 之后重放所有日志

|

(Insert → 写入, Delete → 写入旧值,


```

|      Update → 写入新值, Abort → 立即 Undo)
|
▼
UndoPhase() ← 从 active_txns_ 中仍活跃的事务逐一回滚
|      (沿 prev_lsn 链逆向 Undo)
|
▼
数据库恢复完成

```

8.3) 实现细节

8.3.1) RedoPhase

```

void RedoPhase() {
    for (auto &[lsn, rec] : log_recs_) {
        if (lsn <= persist_lsn_) continue; // 跳过已持久化的日志
        switch (rec->type_) {
            case LogRecType::kInsert:
                data_[rec->ins_key_] = rec->ins_val_;
                active_txns_[rec->txn_id_] = lsn;
                break;
            case LogRecType::kDelete:
                data_[rec->del_key_] = rec->del_val_; // 重放删除：写入空值
                active_txns_[rec->txn_id_] = lsn;
                break;
            case LogRecType::kUpdate:
                data_[rec->new_key_] = rec->new_val_; // 重放更新：写入新值
                active_txns_[rec->txn_id_] = lsn;
                break;
            case LogRecType::kCommit:
                active_txns_.erase(rec->txn_id_); // 提交：从活跃表移除
                break;
            case LogRecType::kAbort:
                UndoTxn(rec->prev_lsn_); // 回滚：立即撤销
                active_txns_.erase(rec->txn_id_);
                break;
        }
    }
}

```

Redo 阶段从 Checkpoint 之后重放所有日志。对于 Insert/Delete/Update 操作，将数据写入 data_ 并记录活跃事务。遇到 Commit 则从活跃事务表移除；遇到 Abort 则立即执行 Undo。

8.3.2) UndoPhase

```

void UndoPhase() {
    for (auto &[txn_id, last_lsn] : active_txns_) {
        UndoTxn(last_lsn);
    }
}

```

```

    active_txns_.clear();
}

```

Undo 阶段对所有在 Redo 结束后仍活跃的事务（未提交/回滚的）执行回滚。

8.3.3) UndoTxn

沿 prev_lsn 链逆序遍历事务的所有日志，逆向执行：

- Insert → 删除（data_.erase(rec->ins_key_)）
- Delete → 恢复旧值（data_[rec->del_key_] = rec->del_val_）
- Update → 恢复旧值（data_[rec->old_key_] = rec->old_val_）

8.3.4) 日志创建辅助函数

FillLsnAndPrev 自动分配递增的 LSN，并维护 prev_lsn_map_ 记录每个事务最后一条日志的 LSN，用于设置下一条日志的 prev_lsn。提供 CreateInsertLog、CreateDeleteLog、CreateUpdateLog、CreateBeginLog、CreateCommitLog、CreateAbortLog 六个工厂函数。

8.4) 测试与分析

恢复管理器的正确性通过 test/recovery/recovery_manager_test.cpp 中的测试验证，覆盖了基本操作日志重放、事务提交/回滚、Checkpoint 恢复、Redo 和 Undo 阶段的完整流程。

8.4.1) 测试结果

```

[-----] 1 test from RecoveryManagerTest
[ RUN      ] RecoveryManagerTest.RecoveryTest
[          OK ] RecoveryManagerTest.RecoveryTest (0 ms)
[-----] 1 test from RecoveryManagerTest (0 ms total)

```

图 11: RecoveryManager 测试通过结果

测试名	测试结果
RecoveryManagerTest	通过 

表 7: RecoveryManager 测试结果

基于测试结果，恢复管理器的实现正确处理了事务的提交、回滚和崩溃恢复，满足了 ARIES 算法的核心要求。